

ARQUITETURAS DE SOFTWARE

Material de Estudo Completo — 35 Questões

Prof. Esp. Leonardo Dias | UniFOA | Sistemas de Informação

Padrões de Design · Docker · Contêineres · Arquiteturas Distribuídas

SUMÁRIO

- 1. Introdução aos Conteúdos Cobrados
- 2. Resumo Teórico Completo
 - 2.1 Padrões de Design (Design Patterns)
 - 2.2 Docker e Contêineres
 - 2.3 Arquiteturas Distribuídas e Microsserviços (Revisão)
- 3. Tabelas-Resumo dos Principais Conceitos
- 4. Questões de Fixação (35 Questões)
- 5. Gabarito Comentado Detalhado
- 6. Principais Erros que os Alunos Cometem
- 7. Revisão Rápida para a Prova
- 8. Mapa Mental (Formato Textual)
- 9. Checklist Final de Revisão

1. INTRODUÇÃO AOS CONTEÚDOS COBRADOS

Este material cobre integralmente os tópicos avaliados na disciplina Arquiteturas de Software, com foco especial nos conteúdos das atividades mais recentes (Padrões de Design e Docker/Contêineres) e revisão dos fundamentos da primeira prova.

Tópicos cobertos na avaliação atual

- Padrões de Design: conceito, benefícios, categorias (Criacionais, Estruturais, Comportamentais)
- Padrões específicos: Factory Method, Adapter, Observer, Singleton, Strategy, Module, Decorator
- Padrões mais usados no frontend JavaScript (React, Angular, Vue)
- Docker e Contêineres: conceito, diferença de VMS, imagens, Dockerfile
- Estrutura de projeto com múltiplos contêineres (Front-end, Back-end, Banco de Dados)
- Docker Compose: função, vantagens e uso
- Volumes e persistência de dados
- Classificação dos contêineres: estático, dinâmico, persistente
- Fluxo de comunicação entre contêineres
- REVISÃO: Microsserviços, MVC, Escalabilidade, Arquiteturas Distribuídas (máximo 2 questões)

2. RESUMO TEÓRICO COMPLETO

2.1 Padrões de Design (Design Patterns)

O que são?

Padrões de design são soluções gerais, reutilizáveis e testadas para problemas recorrentes no desenvolvimento de software. Não são código pronto, mas uma forma de pensar e estruturar o código — análogos a uma receita de bolo: o passo a passo validado que pode ser adaptado a diferentes contextos.

Origem

- Conceito originado na Arquitetura Civil por Christopher Alexander (anos 1970)
- Aplicado ao software pelo Gang of Four (GoF) em 1994 no livro *Design Patterns: Elements of Reusable Object-Oriented Software*

Por que usar? — Benefícios Principais

Benefício	Explicação
Reduz Acoplamento	Componentes dependem menos uns dos outros. Analogia: TV que aceita qualquer controle universal.
Aumenta Reutilização	Soluções aplicáveis em vários contextos. Analogia: carregador USB-C universal.
Facilita Manutenção	Código organizado e previsível. Analogia: fiação em conduítes — fácil de trocar.
Linguagem Comum	Devs se comunicam com precisão. 'Vamos usar Observer aqui' — todos entendem.

Classificação dos Padrões (GoF) — 3 Categorias

Categoria	Foco	Exemplos
Criacionais	Como criar objetos sem acoplamento forte	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Estruturais	Como objetos se relacionam e se organizam	Adapter, Facade, Composite, Proxy, Decorator
Comportamentais	Comunicação e distribuição de responsabilidades	Observer, Strategy, Command, State, Chain of Responsibility

⚠ ATENÇÃO: Factory Method (não apenas 'Factory') pertence à categoria CRIACIONAL. Esta distinção aparece em prova!

Padrões em Detalhes — Os Mais Cobrados

Padrão	Categoria	O que faz	Exemplo prático
Singleton	Criacional	Garante que uma classe tenha somente UMA instância global. Útil para configurações, conexões de banco, logs.	Classe Configuracao que guarda taxa de desconto — qualquer import retorna sempre a mesma instância.
Factory Method	Criacional	Centraliza a criação de objetos de forma padronizada, evitando	fabricaProduto.js — função criarProduto(tipo) retorna

		repetição e escondendo a lógica de construção.	objetos diferentes conforme o tipo.
Adapter	Estrutural	Permite que interfaces incompatíveis trabalhem juntas. Converte a interface de uma classe para outra esperada.	Adaptador de tomada elétrica: plugue americano → tomada brasileira. No software: adaptar diferentes
Facade	Estrutural	Fornecer interface simplificada para um subsistema complexo.	Módulo que expõe apenas os métodos necessários de um sistema grande.
Observer (Pub/Sub)	Comportamental	Define dependência 1:N onde, ao mudar estado, todos os dependentes são notificados automaticamente.	Jornal e assinantes: quando sai nova edição, todos assinantes recebem. No código: addEventListener n
Strategy	Comportamental	Define família de algoritmos intercambiáveis. O cliente escolhe a estratégia em tempo de execução.	estrategiaDesconto.js: DescontoPadrao usa percentual do Singleton; DescontoFidelidade aplica 10% fix
Command	Comportamental	Encapsula uma ação em um objeto executável. Permite enfileirar, registrar ou desfazer operações.	comandoCriarPedido.js — método executar() cria o pedido, salva no repositório e notifica observadore
Repository	Comportamental*	Camada de abstração entre aplicação e fonte de dados. A lógica de negócio não conhece o banco diretamente.	repositorioPedido.js com métodos salvar() e listar(). Se migrar de memória para MongoDB, só o reposi
Module	Comportamental*	Encapsula lógica e evita poluição do escopo global. Hoje implementado com ES Modules (import/export).	Módulo de validação de formulários. Cada arquivo JS é naturalmente um módulo.
Decorator	Estrutural	Adiciona responsabilidades a objetos dinamicamente, sem herança. Em Angular/NestJS: @Component, @Injectable.	Higher-order components (HOC) em React. Decorators em classes TypeScript/NestJS.

Padrões mais usados no Frontend JavaScript

Segundo o material da disciplina, os padrões mais frequentes no frontend (React, Angular, Vue) são:

- Singleton — gerenciador de tema, serviço de autenticação compartilhado
- Observer / Pub-Sub — eventos do DOM (addEventListener), Redux, Vuex
- Strategy — validadores de senha com diferentes regras
- Decorator — @Component e @Injectable no Angular; HOCs e hooks no React
- Module — ES Modules (import/export), encapsulamento de lógica

Resposta da Questão 5: Singleton, Observer, Strategy, Decorator e Module

Diferença: Padrão de Projeto vs. Padrão Arquitetural

Padrão de Projeto (Design Pattern)	Padrão Arquitetural
Foco em nível de código (classes e objetos)	Foco em nível de sistema (módulos, camadas, componentes)

Ex: Observer, Singleton, Factory	Ex: MVC, Microserviços, Camadas
Analogia: design dos móveis dentro dos cômodos	Analogia: planta da casa (como os cômodos se organizam)

2.2 Docker e Contêineres

O que é um Contêiner?

Um contêiner é um ambiente isolado que contém o código da aplicação, suas dependências, configurações e tudo necessário para executá-la corretamente. Diferente das VMs, os contêineres compartilham o kernel do sistema operacional do host, tornando-se mais leves e rápidos.

Contêiner vs. Máquina Virtual

Aspecto	Máquina Virtual	Contêiner	Vantagem
Isolamento	Total (inclui o SO)	Parcial (compartilha o kernel)	—
Inicialização	Lento (segundos/minutos)	Rápido (frações de segundo)	Contêiner
Tamanho	Gigabytes	Megabytes	Contêiner
Uso de recursos	Alto	Baixo	Contêiner
Portabilidade	Limitada	Alta	Contêiner
Ideal para	Infraestruturas tradicionais	Microserviços / DevOps	—

⚠ ATENÇÃO: A característica que explica o tamanho em MB e inicialização rápida dos contêineres é o **COMPARTILHAMENTO DO KERNEL** do SO do host — não é Alpine, não é Kubernetes.

Conceitos Fundamentais do Docker

Conceito	Definição
Imagem	Modelo imutável (receita/molde) que contém tudo o que o contêiner precisa
Contêiner	Instância em execução de uma imagem
Dockerfile	Script de instruções para construir uma imagem
Registry	Repositório de imagens (ex: Docker Hub)
Docker Compose	Ferramenta para definir e executar múltiplos contêineres de forma integrada
Volume	Mecanismo de persistência de dados que sobrevive ao ciclo de vida do contêiner

Arquitetura de 3 Contêineres (Projeto Completo)

Um projeto completo com Front-end, Back-end e Banco de Dados possui 3 contêineres, cada um com papel distinto:

Contêiner	Imagem Base	Classificação	Usa Volume?	Função Principal
Front-End	nginx:alpine	ESTÁTICO	Não necessário	Servir arquivos HTML/CSS/JS ao navegador
Back-End	node:alpine	DINÂMICO / LÓGICO	Opcional	Processar regras de negócio e responder à API

Banco de Dados	mongo/mysql/postgres	PERSISTENTE	FUNDAMENTAL	Armazenar e recuperar dados da aplicação
----------------	----------------------	-------------	-------------	--

Por que o Banco de Dados precisa de Volumes?

Contêineres são efêmeros: quando são desligados ou recriados, seus dados internos se perdem. O volume mapeia um diretório do host para dentro do contêiner, garantindo que os dados do banco sejam persistidos mesmo após reinicializações.

Volumes = garantia de que os dados não se percam quando o contêiner é desligado ou recriado.

Docker Compose — Função Principal

O Docker Compose permite definir e executar múltiplos contêineres de uma aplicação ao mesmo tempo, descrevendo toda a estrutura em um único arquivo (docker-compose.yml). Ao iniciar, cria automaticamente: os contêineres configurados E a rede interna para comunicação entre eles.

⚠ ATENÇÃO: Docker Compose NÃO gerencia recursos de produção como CPU/memória (isso é Kubernetes). NÃO armazena imagens no Registry (isso é docker push/pull). NÃO fornece Nginx+Node em um único contêiner.

Fluxo de Comunicação entre os 3 Contêineres

Fluxo correto de uma requisição completa:

1. Usuário interage com o Front-end (interface no navegador)
2. Front-end envia requisição HTTP ao Back-end (via rede interna Docker)
3. Back-end processa as regras de negócio e consulta o Banco de Dados
4. Banco de Dados retorna os dados ao Back-end
5. Back-end devolve a resposta ao Front-end
6. Front-end exibe o resultado ao usuário

IMPORTANTE: O Front-end NUNCA se comunica diretamente com o Banco de Dados. O Back-end é sempre o intermediário.

2.3 Revisão — Arquiteturas Distribuídas e Microsserviços

(Conteúdo da 1ª prova — máximo 2 questões de revisão incluídas no simulado)

Conceito	Definição Rápida
Arquitetura de Software	Estrutura fundamental de um sistema: componentes, relações e princípios que guiam seu projeto e evolução.
Modularidade	Divisão em partes coesas e fracamente acopladas.
Escalabilidade Vertical	Mais CPU/memória na mesma máquina.
Escalabilidade Horizontal	Adicionar novas instâncias (Docker/Kubernetes).
MVC	Model (dados+lógica), View (interface), Controller (intermediário). Regras de negócio ficam no MODEL.
Microsserviços	Serviços pequenos, independentes, comunicam-se via APIs. Escala granular.
Teorema CAP	Consistência, Disponibilidade, Tolerância a Partições — só 2 ao mesmo tempo.
Circuit Breaker	Evita falhas em cascata interrompendo chamadas a serviços com problema.
API Gateway	Ponto central que organiza e roteia o acesso aos microsserviços.
SOA	Precursor dos microsserviços. Serviços independentes com interfaces bem definidas.
Monorepo	Todos os serviços no mesmo repositório. Facilita CI/CD unificado.
Multirepo	Cada serviço em repositório separado. Maior autonomia por serviço.

3. TABELAS-RESUMO DOS PRINCIPAIS CONCEITOS

Classificação dos Padrões de Design (GoF)

Padrão	Categoria	Analogia
Singleton	Criacional	Um único gerente que toma todas as decisões da empresa
Factory Method	Criacional	Chef de cozinha que prepara pratos diferentes sob demanda
Builder	Criacional	Construtor que monta uma casa etapa por etapa
Adapter	Estrutural	Adaptador de tomada elétrica (plugue incompatível)
Facade	Estrutural	Painel de controle único para sistema complexo
Decorator	Estrutural	@Anotações em classes; HOC no React
Observer	Comportamental	Jornal e assinantes — notificação automática
Strategy	Comportamental	GPS com diferentes rotas — troca o algoritmo em tempo real
Command	Comportamental	Botão desfazer (Ctrl+Z) — ação encapsulada em objeto
Repository	Comportamental*	Balcão de atendimento — a aplicação não sabe de onde vêm os dados

Comparativo: Contêiner vs. VM vs. Bare Metal

Aspecto	Bare Metal	VM	Contêiner
Isolamento	Nenhum	Total (SO próprio)	Parcial (kernel compartilhado)
Tamanho	—	Gigabytes	Megabytes
Inicialização	Minutos	Segundos/Minutos	Frações de segundo
Portabilidade	Baixa	Média	Alta
Uso ideal	Aplicações legadas	Infraestrutura tradicional	Microsserviços / DevOps

4. QUESTÕES DE FIXAÇÃO — 35 QUESTÕES

Instruções: selecione a alternativa correta em cada questão. O gabarito comentado está na seção 5.

Questão 1 —

Qual das opções representa um benefício direto do uso de padrões de design?

- A) Aumento do tamanho do código-fonte.
- B) Dificuldade na manutenção do sistema.
- C) Redução do acoplamento entre componentes.**
- D) Maior dependência entre classes e objetos.

☒ **Gabarito:** C

Explicação: Padrões de design são projetados exatamente para reduzir o acoplamento, tornando os componentes mais independentes e fáceis de substituir ou modificar. As alternativas A, B e D descrevem problemas — exatamente o oposto do que os padrões resolvem.

✗ Por que as outras estão erradas: A: padrões costumam reduzir código repetido, não aumentar; B: facilitam a manutenção; D: reduzem, não aumentam dependências.

💡 Pense: padrões = soluções para problemas. As opções erradas descrevem PROBLEMAS, não benefícios.

Questão 2 —

O Factory Method é um padrão pertencente a qual categoria?

- A) Estrutural
- B) Comportamental
- C) Arquitetural
- D) Criacional**

☒ **Gabarito:** D

Explicação: O Factory Method é um padrão Criacional — foca em como criar objetos sem acoplamento forte entre quem cria e o que é criado. A categoria Criacional inclui: Singleton, Factory Method, Abstract Factory, Builder e Prototype.

✗ Por que as outras estão erradas: A: Estrutural é sobre composição de objetos (Adapter, Facade...); B: Comportamental é sobre comunicação (Observer, Strategy...); C: 'Arquitetural' não é uma das 3 categorias GoF.

💡 Mnemônico: CEC — Criacional cria, Estrutural estrutura, Comportamental comporta. Factory = cria objetos = Criacional.

Questão 3 —

O que melhor define um padrão de design?

- A) Uma solução genérica e testada para problemas recorrentes de desenvolvimento.**
- B) Um código reutilizável criado apenas para linguagens orientadas a objetos.
- C) Um modelo de interface gráfica pronto para uso.
- D) Um algoritmo que resolve qualquer problema de software automaticamente.

✓ **Gabarito:** A

📖 **Explicação:** Padrões de design são soluções gerais, reutilizáveis e testadas para problemas recorrentes — não são código pronto, mas uma forma de pensar e estruturar o código, aplicáveis em diferentes linguagens e contextos.

✗ **Por que as outras estão erradas:** B: não são exclusivos de OO, embora sejam muito usados nesse paradigma; C: não são modelos de UI; D: não resolvem 'qualquer problema' automaticamente.

💡 A palavra-chave da definição clássica do GoF: 'solução geral e testada para problemas recorrentes'.

Questão 4 —

Qual analogia representa corretamente o papel de um Adapter Pattern?

A) Um jornal que envia notícias aos seus assinantes.

B) Um adaptador de tomada que permite conectar plugues de diferentes padrões.

C) Um livro de receitas que ensina a preparar diferentes bolos.

D) Um controle remoto universal que substitui o original da TV.

✓ **Gabarito:** B

📖 **Explicação:** O Adapter é um padrão Estrutural que permite que interfaces incompatíveis trabalhem juntas — exatamente como um adaptador de tomada que permite conectar plugues de padrões diferentes (ex: americano em tomada brasileira).

✗ **Por que as outras estão erradas:** A: descreve o Observer (publicador e assinantes); C: descreve o Factory ou Template Method; D: descreve redução de dependência/acoplamento em geral.

💡 Adapter = adaptador físico. Incompatibilidade → compatibilidade. Observer = jornal + assinantes.

Questão 5 —

Quais padrões de design são mais utilizados no frontend JavaScript?

A) Adapter, Facade, Prototype e Builder

B) Singleton, Observer, Strategy, Decorator e Module

C) Factory, Repository, Chain of Responsibility e State

D) Repository, Adapter, Proxy e Command

✓ **Gabarito:** B

📖 **Explicação:** No frontend JavaScript (React, Angular, Vue), os padrões mais frequentes são: Singleton (serviços compartilhados), Observer/Pub-Sub (eventos do DOM, Redux), Strategy (validadores), Decorator (@Component, HOCs) e Module (ES Modules).

✗ **Por que as outras estão erradas:** A: Adapter e Facade são estruturais mas não os mais típicos do frontend; C e D: esses padrões são mais comuns no backend Node.js.

💡 SSDOM: Singleton, Strategy, Decorator, Observer, Module — os 5 do frontend JS.

Questão 6 —

Na arquitetura de três contêineres (Front-End, Back-End e Banco de Dados), o contêiner de Banco de Dados é classificado como Persistente. Por que o uso de volumes é considerado fundamental para este tipo de contêiner, diferentemente dos outros?

A) Porque os volumes garantem que as informações e os registros (dados) não se percam quando o contêiner é desligado ou recriado.

B) Porque o Front-end armazena em volumes o código-fonte da aplicação antes que ele seja servido pelo Nginx.

C) Porque o Banco de Dados precisa de volumes para se comunicar com o Back-end, garantindo a integridade da conexão.

D) Porque o Back-end necessita de um volume para salvar os logs e o Banco de Dados depende desses logs para funcionar.

✓ **Gabarito: A**

📖 **Explicação:** Contêineres são efêmeros: ao serem removidos ou recriados, seus dados internos se perdem. O volume é um mapeamento para o host que persiste os dados do banco independentemente do ciclo de vida do contêiner. Isso é fundamental para banco de dados, pois dados são valiosos e não podem ser perdidos.

✗ **Por que as outras estão erradas:** B: o Nginx serve arquivos estáticos já presentes na imagem; C: volumes não são usados para comunicação entre contêineres (isso é feito pela rede interna Docker); D: logs do back-end são independentes do banco.

💡 *Volume = persistência. Contêiner sem volume = dado temporário que some ao desligar. Banco precisa de dado permanente.*

Questão 7 —

Qual característica fundamental dos contêineres, em comparação com as Máquinas Virtuais (VMs), contribui diretamente para seu tamanho reduzido (Megabytes) e tempo de inicialização rápido?

A) A exigência de imagens baseadas em distribuições Linux leves, como a versão Alpine, para todas as aplicações.

B) A exclusividade para uso em arquiteturas baseadas em microsserviços, que naturalmente utilizam menos recursos que infraestruturas tradicionais.

C) O compartilhamento do kernel do sistema operacional do host, o que elimina a necessidade de incluir um sistema operacional completo em cada instância.

D) O uso de ferramentas de orquestração como o Kubernetes, que otimiza o uso de recursos de rede e CPU.

✓ **Gabarito: C**

📖 **Explicação:** A diferença fundamental é que contêineres COMPARTILHAM o kernel do SO do host. VMs precisam de um SO completo (kernel próprio) em cada instância, por isso são da ordem de Gigabytes. Contêineres precisam apenas da aplicação e suas dependências, por isso são Megabytes e iniciam em frações de segundo.

✗ **Por que as outras estão erradas:** A: Alpine é opcional, não obrigatório; contêineres continuam mais leves mesmo com imagens non-Alpine; B: contêineres não são exclusivos de microsserviços; D: Kubernetes é orquestração, não responsável pelo tamanho da imagem.

💡 *VM = apartamento com cozinha própria. Contêiner = quarto em república compartilhando cozinha (kernel). Por isso o contêiner é menor e mais rápido!*

Questão 8 —

Em um projeto que utiliza Docker para containerizar um Front-end e um Back-end, qual é a principal função da ferramenta Docker Compose?

A) Definir e executar múltiplos contêineres de uma aplicação de forma integrada (ex: Back-end e Front-end), criando automaticamente a rede interna para comunicação entre eles.

B) Fornecer o servidor web Nginx para o Front-end e o interpretador Node.js para o Back-end em um único contêiner unificado.

C) Gerenciar a alocação de recursos de CPU, memória e armazenamento para os contêineres em produção, característica do Kubernetes.

D) Construir as imagens do contêiner a partir dos Dockerfiles e armazená-las no Registry (ex: Docker Hub).

✓ **Gabarito:** A

📖 **Explicação:** O Docker Compose orquestra múltiplos contêineres em conjunto, descrevendo toda a estrutura no arquivo docker-compose.yml. Ao iniciar, cria automaticamente os contêineres configurados e a rede interna que permite comunicação entre eles.

✗ **Por que as outras estão erradas:** B: cada contêiner tem sua própria imagem, não são unificados; C: gerenciamento de recursos em produção é função do Kubernetes, não do Compose; D: construir e armazenar imagens é função do docker build e docker push/Registry.

💡 *Compose = orquestração SIMPLES (dev/test). Kubernetes = orquestração COMPLEXA (produção em escala).*

Questão 9 —

A imagem de contêiner utilizada para o Front-End é tipicamente baseada em Nginx (por exemplo, nginx:alpine). Qual das alternativas descreve corretamente o seu papel principal e a sua classificação?

A) É classificada como Dinâmica, sendo responsável por executar o código JavaScript no servidor e aplicar as regras de negócio.

B) É classificada como Lógica e Dinâmica, mantendo um processo ativo para processar requisições e consultar o banco de dados.

C) É classificada como Persistente, sendo o único contêiner que necessita de volumes para garantir a integridade dos dados.

D) É classificada como Estática, sendo responsável por servir os arquivos estáticos da interface (HTML, CSS e JavaScript) ao navegador do usuário.

✓ **Gabarito:** D

📖 **Explicação:** O Nginx é um servidor web que distribui arquivos estáticos (HTML, CSS, JS já construídos). Não executa lógica de negócio. Por isso o Front-end é classificado como ESTÁTICO — entrega conteúdo pronto ao navegador, sem processar requisições dinâmicas.

✗ **Por que as outras estão erradas:** A e B: descrevem o Back-end (Node.js), que é dinâmico e processa lógica; C: o contêiner persistente é o Banco de Dados, não o Front-end.

💡 *Front = Nginx = Estático. Back = Node.js = Dinâmico. BD = mongo/mysql = Persistente. Memorize os 3 perfis!*

Questão 10 —

Em uma transação completa, onde o usuário interage com o Front-end e este precisa consultar dados no Banco de Dados, qual é o fluxo de comunicação e responsabilidade correto entre os contêineres?

A) Front-end (envia requisição) → Banco de Dados (consulta direta), pois ambos são acessíveis externamente ao usuário.

B) Front-end (envia requisição) → Back-end (processa regras e consulta o DB) → Banco de Dados (retorna dados), e o resultado final é devolvido ao Front-end via Back-end.

C) Back-end (requisição) → Front-end (apresentação) → Banco de Dados (armazenamento).

D) Front-end (requisição) → Banco de Dados (consulta) → Back-end (processamento).

✓ **Gabarito:** B

📖 **Explicação:** O fluxo correto é: Front-end → Back-end → Banco de Dados → Back-end → Front-end. O Back-end é SEMPRE o intermediário entre o Front-end e o Banco de Dados. O banco de dados não se comunica diretamente com o front-end por questões de segurança, arquitetura e responsabilidade de camadas.

✗ **Por que as outras estão erradas:** A: front-end nunca acessa diretamente o banco — isso seria uma falha grave de segurança; C: a sequência está invertida; D: o banco não processa regras de negócio.

💡 *Regra de ouro: Front → Back → Banco. Nunca pule o Back-end! É como um cliente que só fala com o garçom, nunca vai direto à cozinha.*

Questão 11 —

No padrão Observer, quando um evento ocorre no publicador (subject), o que acontece automaticamente com todos os seus observadores inscritos?

A) Eles são destruídos e recriados com o novo estado.

B) Eles são notificados automaticamente e podem reagir ao evento de forma independente.

C) Apenas o primeiro observador inscrito recebe a notificação, em ordem de prioridade.

D) Eles precisam consultar periodicamente o publicador para verificar se houve mudança.

✓ **Gabarito:** B

📖 **Explicação:** O Observer (Pub/Sub) é um padrão comportamental que define dependência 1:N: quando o estado do publicador muda, TODOS os observadores inscritos são notificados automaticamente e cada um pode reagir de forma independente. O acoplamento é baixo pois o publicador não conhece os observadores individualmente.

✗ **Por que as outras estão erradas:** A: observadores não são destruídos; C: todos os inscritos recebem (não só o primeiro); D: a consulta periódica (polling) é o oposto do Observer — no Observer a notificação é PUSH, não PULL.

💡 *Observer = PUSH automático. Polling = PULL manual. No jornal, a editora envia para os assinantes — não os assinantes que ligam perguntando se saiu nova edição!*

Questão 12 —

Qual padrão de design garante que exista apenas uma instância de uma classe em toda a aplicação, fornecendo um ponto de acesso global a essa instância?

- A) Factory Method
- B) Observer
- C) Singleton**
- D) Repository
- E) Strategy

✓ **Gabarito:** C

📖 **Explicação:** O Singleton é um padrão Criacional que restringe a instanciação de uma classe a um único objeto. No código: o construtor verifica se já existe uma instância e retorna ela em vez de criar nova. Útil para configurações globais, conexões de banco, gerenciadores de log.

✗ **Por que as outras estão erradas:** A: Factory cria objetos, mas pode criar múltiplos; B: Observer gerencia notificações; D: Repository abstrai acesso a dados; E: Strategy troca algoritmos em tempo de execução.

💡 *Singleton = 'único' (single). Se você precisar garantir 'só um existe', pense Singleton.*

Questão 13 —

No projeto pedido-simples, o padrão Strategy foi implementado para o cálculo de descontos. Qual é o principal benefício desse padrão nesse contexto?

- A) Garantir que apenas um desconto seja calculado por pedido, evitando duplicidades.
- B) Permitir trocar o algoritmo de cálculo de desconto em tempo de execução sem modificar o código que consome o resultado.**
- C) Armazenar os descontos aplicados em um banco de dados para auditoria posterior.
- D) Centralizar todas as regras de desconto em um único arquivo impossível de modificar.

✓ **Gabarito:** B

📖 **Explicação:** O Strategy permite isolar comportamentos intercambiáveis. No projeto: DescontoPadrao e DescontoFidelidade implementam a mesma interface, e o app.js pode escolher qual usar em tempo de execução sem saber os detalhes internos de cada algoritmo. Isso facilita adicionar novos tipos de desconto sem tocar no código existente.

✗ **Por que as outras estão erradas:** A: não é sobre evitar duplicidades; C: armazenamento é papel do Repository; D: Strategy promove FLEXIBILIDADE, não rigidez.

💡 *Strategy = trocar algoritmo em tempo de execução. Pense em GPS: você muda a rota (algoritmo) sem mudar o destino (objetivo).*

Questão 14 —

O padrão Repository tem como objetivo principal:

- A) Garantir que exista apenas uma conexão com o banco de dados em toda a aplicação.
- B) Criar objetos de diferentes tipos a partir de uma função ou método centralizado.
- C) Servir como camada intermediária entre a aplicação e a fonte de dados, isolando o acesso ao armazenamento.**
- D) Notificar componentes interessados quando o estado dos dados muda.

✓ **Gabarito:** C

 **Explicação:** O Repository abstrai o acesso a dados: a lógica de negócio usa métodos como salvar() e listar() sem se importar se os dados estão em memória, arquivo ou banco relacional. Permite trocar a implementação (ex: de memória para MongoDB) sem mudar a lógica de negócio.

 **Por que as outras estão erradas:** A: isso é Singleton; B: isso é Factory; D: isso é Observer.

 *Repository = balcão de pedidos. A cozinha (lógica) pede sem saber de onde vem o ingrediente (DB).*

Questão 15 —

Qual das alternativas descreve corretamente a diferença entre a imagem do contêiner de Back-End (Node.js) e a imagem do contêiner de Front-End (Nginx)?

A) Ambas são imagens dinâmicas, a diferença é apenas a linguagem de programação utilizada.

B) A imagem do Back-End é dinâmica e executa código JavaScript no servidor; a do Front-End é estática e serve arquivos construídos ao navegador.

C) A imagem do Front-End é dinâmica porque o Nginx processa requisições em tempo real; a do Back-End é estática pois não usa banco de dados.

D) A imagem do Back-End é persistente e necessita de volumes; a do Front-End é dinâmica.

 **Gabarito:** B

 **Explicação:** Back-End (node:alpine): dinâmico, mantém processo ativo, processa requisições, aplica regras de negócio, acessa banco. Front-End (nginx:alpine): estático, distribui os arquivos já construídos (HTML/CSS/JS), não processa lógica de aplicação. Essa distinção é fundamental para classificar os contêineres.

 **Por que as outras estão erradas:** A: não são ambas dinâmicas; C: Nginx não é dinâmico — ele serve arquivos; D: persistente é o banco de dados.

 *Node = processa = dinâmico. Nginx = entrega = estático. Banco = guarda = persistente. Três palavras para três imagens!*

Questão 16 —

Em uma aplicação containerizada com Docker Compose, como os contêineres de Front-end e Back-end se comunicam entre si?

A) Por meio de endereços IP externos (públicos) expostos ao usuário final.

B) Através de uma rede interna criada automaticamente pelo Docker, onde cada contêiner é identificado pelo seu nome no arquivo docker-compose.yml.

C) Apenas por meio de variáveis de ambiente compartilhadas no mesmo arquivo .env.

D) Por chamadas diretas ao sistema operacional host, sem intermediários.

 **Gabarito:** B

 **Explicação:** O Docker (e o Docker Compose) cria automaticamente uma rede interna privada. Dentro dessa rede, cada contêiner é identificado pelo nome definido no docker-compose.yml (ex: 'api' e 'front'). O front-end pode chamar http://api:3000 para se comunicar com o back-end sem usar IPs externos.

 **Por que as outras estão erradas:** A: IPs externos não são necessários para comunicação interna entre contêineres; C: variáveis de ambiente configuram, mas não são o canal de comunicação HTTP; D: contêineres não fazem chamadas diretas ao SO host.

💡 Rede interna Docker = DNS automático. O nome do serviço no Compose vira o hostname na rede.

Questão 17 —

Qual dos seguintes NÃO é um conceito fundamental do Docker?

- A) Imagem — modelo imutável que contém tudo o que o contêiner precisa
- B) Dockerfile — script de instruções para construir uma imagem
- C) Pod — unidade básica de agendamento de contêineres no Docker**
- D) Registry — repositório de imagens como o Docker Hub

✅ **Gabarito:** C

📖 **Explicação:** Pod é um conceito do KUBERNETES, não do Docker. No Kubernetes, um Pod é a menor unidade de implantação e pode conter um ou mais contêineres. No Docker puro, os conceitos fundamentais são: Imagem, Contêiner, Dockerfile e Registry.

❌ **Por que as outras estão erradas:** A, B e D: são todos conceitos corretos e fundamentais do Docker.

💡 Pod = Kubernetes. Imagem/Contêiner/Dockerfile/Registry = Docker. Não confunda as ferramentas!

Questão 18 —

O padrão Command encapsula uma ação em um objeto. Qual é uma das principais vantagens dessa abordagem?

- A) Eliminar a necessidade de banco de dados para armazenar o histórico de operações.
- B) Permitir enfileirar, registrar histórico e potencialmente desfazer (undo) operações com facilidade.**
- C) Garantir que apenas um comando seja executado por vez em toda a aplicação.
- D) Forçar que todas as operações sejam síncronas para garantir a ordem de execução.

✅ **Gabarito:** B

📖 **Explicação:** O Command transforma operações em objetos, o que permite: enfileirar comandos para execução posterior (workers/filas), registrar histórico para auditoria, implementar undo/redo e enviar comandos para outros sistemas. No projeto pedido-simples, o comando CriarPedido.js encapsula criar+salvar+notificar em um único objeto.

❌ **Por que as outras estão erradas:** A: o banco ainda é necessário; C: não há restrição de execução única; D: Command pode ser síncrono ou assíncrono.

💡 Command = Ctrl+Z habilitado. Se a ação é um objeto, você pode guardá-la, repeti-la, ou desfazê-la!

Questão 19 —

No contexto do padrão de projeto Adapter, qual problema ele resolve?

- A) Garante que apenas uma instância de um adaptador exista na aplicação.
- B) Permite que classes com interfaces incompatíveis trabalhem juntas sem modificar o código original.**
- C) Adiciona novas funcionalidades a uma classe existente por meio de herança múltipla.

D) Cria objetos de diferentes tipos a partir de um método centralizado.

✓ **Gabarito:** B

📖 **Explicação:** O Adapter é um padrão Estrutural que converte a interface de uma classe em outra esperada pelo cliente. Permite integrar sistemas com APIs diferentes sem reescrever o código existente. Ex: adaptar diferentes provedores de pagamento (PayPal, Stripe) para uma interface única na aplicação.

✗ **Por que as outras estão erradas:** A: isso é Singleton; C: Decorator adiciona funcionalidades dinamicamente, não Adapter; D: isso é Factory.

💡 *Adapter = plug converter. Código antigo + nova API = Adapter no meio para fazer funcionar juntos.*

Questão 20 —

No ecossistema JavaScript backend (Node.js/NestJS), qual padrão é tipicamente usado para evitar criar múltiplas conexões com o banco de dados e garantir reuso da conexão existente?

A) Observer

B) Command

C) Singleton

D) Repository

E) Factory

✓ **Gabarito:** C

📖 **Explicação:** O Singleton garante que apenas uma instância exista. No backend Node.js, a conexão com o banco (MongoDB, PostgreSQL) é um recurso caro. O Singleton garante que todos os módulos reutilizem a mesma conexão já estabelecida, evitando criar dezenas de conexões desnecessárias.

✗ **Por que as outras estão erradas:** A: Observer gerencia eventos/notificações; B: Command encapsula ações; D: Repository abstrai acesso a dados mas não gerencia a conexão; E: Factory cria objetos variados.

💡 *Singleton no backend = uma conexão de banco compartilhada por todos. Economiza recursos!*

Questão 21 —

Qual das alternativas melhor descreve o padrão Decorator?

A) Substitui uma classe por outra com interface compatível para adicionar funcionalidades.

B) Adiciona responsabilidades a objetos de forma dinâmica, sem usar herança direta, por meio de envolvimento (wrapping).

C) Garante que objetos de uma família sejam criados de forma coesa e compatível.

D) Define uma interface para criação de objetos, deixando as subclasses decidirem quais instanciar.

✓ **Gabarito:** B

📖 **Explicação:** O Decorator 'embrulha' um objeto com funcionalidades adicionais. No Angular/NestJS, @Component e @Injectable são decorators que adicionam metadados às classes. Em React, HOCs (Higher-Order Components) são a expressão do Decorator. A vantagem: extensibilidade sem herança.

✗ **Por que as outras estão erradas:** A: descreve Adapter (compatibilidade de interface); C: descreve Abstract Factory (família de objetos); D: descreve Factory Method.

💡 Decorator = presente embrulhado. O objeto original está lá, mas com camadas extras de funcionalidade ao redor.

Questão 22 —

Qual é a diferença fundamental entre Docker Compose e Kubernetes?

A) Docker Compose cria imagens; Kubernetes executa contêineres.

B) Docker Compose é usado para orquestração simples em desenvolvimento/testes; Kubernetes é para orquestração complexa em produção com escala.

C) Docker Compose só suporta contêineres Linux; Kubernetes suporta qualquer sistema operacional.

D) Docker Compose cria redes internas; Kubernetes não suporta redes entre contêineres.

✓ **Gabarito:** B

📖 **Explicação:** Docker Compose é ideal para definir e executar múltiplos contêineres localmente (desenvolvimento, testes, CI). Kubernetes é a ferramenta de orquestração de produção, gerenciando centenas/milhares de contêineres com features como auto-scaling, self-healing, rolling updates e balanceamento de carga avançado.

✗ **Por que as outras estão erradas:** A: ambos podem usar imagens existentes; C: ambos suportam Linux como base; D: Kubernetes também cria redes entre Pods.

💡 Compose = orquestração para o desenvolvedor. Kubernetes = orquestração para o operador de produção.

Questão 23 —

No projeto pedido-simples, qual é o papel do arquivo app.js em relação aos padrões de design implementados?

A) Implementa o padrão Singleton para garantir uma única instância da aplicação.

B) Define as estratégias de desconto que serão usadas pelos outros módulos.

C) Orquestra todos os padrões em funcionamento, criando produtos, aplicando descontos, montando pedidos e executando os comandos.

D) Atua como repositório central de dados, armazenando os pedidos criados.

✓ **Gabarito:** C

📖 **Explicação:** O app.js é o 'maestro' da aplicação: (1) usa Factory para criar produtos; (2) usa Strategy para aplicar descontos; (3) monta o pedido; (4) executa o Command que salva no Repository e dispara o Observer. Ele orquestra todos os padrões sem implementar nenhum deles diretamente.

✗ **Por que as outras estão erradas:** A: o Singleton está em configuracao.js; B: as estratégias estão em estrategiaDesconto.js; D: o repositório está em repositorioPedido.js.

💡 app.js = maestro da orquestra. Cada músico (padrão) sabe seu papel; o maestro coordena todos.

Questão 24 —

Um desenvolvedor percebe que, ao adicionar um novo tipo de produto ao sistema, precisou alterar código em 10 arquivos diferentes. Qual princípio/padrão, se aplicado, reduziria essa necessidade?

A) Observer — para notificar todos os módulos quando um produto novo for criado.

B) Factory — para centralizar a criação de produtos em um único lugar, evitando que a lógica de construção se espalhe pelo código.

C) Singleton — para garantir que apenas um produto seja criado por vez.

D) Command — para encapsular a criação de produto em um objeto executável.

✓ **Gabarito: B**

📖 **Explicação:** O Factory centraliza a lógica de criação. Se a criação de produtos está espalhada por 10 arquivos, adicionar um novo tipo exige alterar todos os 10. Com Factory, a lógica de criação fica em um único lugar (fabricaProduto.js); adicionar novo tipo = alterar apenas o Factory. Isso aplica o princípio Open/Closed (aberto para extensão, fechado para modificação).

✗ **Por que as outras estão erradas:** A: Observer é para notificações, não criação; C: Singleton é para instância única, não centralização de criação de tipos; D: Command encapsula ações, não cria tipos de objetos.

💡 'Preciso alterar muitos lugares para criar um tipo' = sinal de que precisa de Factory!

Questão 25 —

Qual afirmação descreve corretamente a relação entre o padrão Singleton e o padrão Strategy no projeto pedido-simples?

A) O Strategy cria uma instância do Singleton para cada estratégia de desconto aplicada.

B) O Singleton (configuracao.js) fornece o percentual de desconto padrão que a classe DescontoPadrao (Strategy) utiliza.

C) O Singleton e o Strategy são independentes e não se relacionam no projeto.

D) O Strategy gerencia a instância única do Singleton para garantir consistência.

✓ **Gabarito: B**

📖 **Explicação:** No projeto pedido-simples há uma relação direta: o configuracao.js (Singleton) armazena o percentualDescontoPadrao. O estrategiaDesconto.js (Strategy) — especificamente a classe DescontoPadrao — consulta essa configuração do Singleton para calcular o desconto. O Singleton fornece o dado; o Strategy define como usá-lo.

✗ **Por que as outras estão erradas:** A: o Strategy não cria instâncias do Singleton; C: eles se relacionam diretamente; D: o Strategy não gerencia o Singleton.

💡 No mapa de dependências: Singleton → (fornece config para) → Strategy. Memorize: Singleton alimenta Strategy.

Questão 26 —

Em qual situação o uso do padrão Facade é mais indicado?

A) Quando é necessário garantir que apenas uma instância de uma classe exista no sistema.

B) Quando um subsistema é complexo e precisa de uma interface simplificada para os clientes.

C) Quando diferentes algoritmos intercambiáveis precisam ser aplicados ao mesmo problema.

D) Quando é necessário notificar múltiplos componentes sobre mudanças de estado.

✓ **Gabarito:** B

📖 **Explicação:** O Facade fornece uma interface simplificada para um subsistema complexo. Em monólitos, o Facade evita que o código fique 'espaguete', expondo apenas os métodos essenciais de um módulo grande. Ex: uma classe EmailService que internamente usa SMTP, autenticação, templates e filas — mas expõe apenas enviar(destinatario, mensagem).

✗ **Por que as outras estão erradas:** A: isso é Singleton; C: isso é Strategy; D: isso é Observer.

💡 *Facade = fachada de prédio. Você vê só a entrada bonita, não toda a estrutura interna.*

Questão 27 —

Qual das alternativas a seguir NÃO representa uma boa prática no uso de contêineres Docker?

- A) Criar imagens pequenas, minimizando o número de camadas e usando versões alpine.
- B) Usar variáveis de ambiente para configuração, evitando informações fixas no Dockerfile.
- C) Armazenar credenciais de banco de dados diretamente no código da imagem para facilitar o acesso.**
- D) Versionar o Dockerfile e o docker-compose.yml no repositório de código.

✓ **Gabarito:** C

📖 **Explicação:** Armazenar credenciais diretamente no código ou na imagem é uma grave falha de segurança. Quem tiver acesso à imagem terá acesso às credenciais. A prática correta é usar variáveis de ambiente, arquivos .env (não versionados) ou sistemas de gerenciamento de segredos (Docker Secrets, Vault).

✗ **Por que as outras estão erradas:** A, B e D: são todas boas práticas recomendadas para o uso de Docker.

💡 *Nunca credencial hardcoded em imagem! Use variáveis de ambiente ou Docker Secrets.*

Questão 28 —

Qual é a principal diferença entre uma Imagem Docker e um Contêiner Docker?

- A) A imagem é executável; o contêiner é apenas um arquivo de configuração.
- B) A imagem é um modelo imutável (molde); o contêiner é uma instância em execução desse molde.**
- C) A imagem armazena os dados da aplicação; o contêiner armazena o código.
- D) A imagem é criada pelo Docker Compose; o contêiner é criado pelo Dockerfile.

✓ **Gabarito:** B

📖 **Explicação:** A imagem é o 'molde' — imutável, contém código, dependências e configurações. O contêiner é a 'instância em execução' da imagem. Uma imagem pode gerar múltiplos contêineres. Analogia: a imagem é a classe (molde); o contêiner é o objeto (instância em execução).

✗ **Por que as outras estão erradas:** A: a imagem não é diretamente executável — ela origina contêineres; C: ambos têm código; D: o Dockerfile cria a imagem, o docker run/Compose cria o contêiner.

💡 *Imagem : Contêiner = Classe : Objeto = Forma de bolo : Bolo assado. O molde (imagem) pode fazer vários bolos (contêineres)!*

Questão 29 —

No padrão arquitetural MVC, onde devem ser implementadas as regras de negócio, como cálculo de desconto e validação de CPF?

- A) Na View, para que o resultado apareça imediatamente na interface.
- B) No Controller, pois ele coordena toda a aplicação.
- C) No Model, que representa dados e lógica de negócio.**
- D) No Banco de Dados, em stored procedures, para centralizar a lógica.

✓ **Gabarito:** C

📖 **Explicação:** No MVC, o Model é responsável pelos dados E pela lógica de negócio. Métodos como calcularDesconto() e validarCPF() pertencem ao Model. Colocar lógica no Controller gera o anti-pattern 'Fat Controller / God Object', que dificulta testes e manutenção.

✗ **Por que as outras estão erradas:** A: View só apresenta dados, sem lógica; B: Controller apenas coordena, não deve ter lógica de negócio complexa; D: stored procedures acoplam o sistema ao banco e dificultam portabilidade.

💡 *Model = dado + regra. View = apresentação. Controller = coordenador. Regra de negócio = sempre no Model!*

Questão 30 —

Qual é o principal desafio das arquiteturas distribuídas relacionado ao Teorema CAP?

- A) A dificuldade de escrever código orientado a objetos em ambientes distribuídos.
- B) A impossibilidade de garantir simultaneamente Consistência, Disponibilidade e Tolerância a Partições em um sistema distribuído.**
- C) O alto custo de licenciamento de ferramentas de monitoramento distribuído.
- D) A necessidade de usar apenas bancos de dados relacionais em sistemas distribuídos.

✓ **Gabarito:** B

📖 **Explicação:** O Teorema CAP (Eric Brewer, 2000) afirma que sistemas distribuídos não podem garantir simultaneamente os 3 atributos: Consistência (todos os nós veem os mesmos dados), Disponibilidade (toda requisição recebe resposta) e Tolerância a Partições (sistema funciona mesmo com falha de comunicação). É necessário escolher 2 dos 3.

✗ **Por que as outras estão erradas:** A: OOP não tem relação com o CAP; C: custo de ferramentas não é o tema; D: bancos NoSQL são comuns em distribuídos justamente pela flexibilidade com CAP.

💡 *CAP = C+A+P, só 2 ao mesmo tempo. SQL ≈ CA. MongoDB ≈ AP. Essa questão aparece em provas de arquitetura!*

Questão 31 —

[REVISÃO] Qual é a principal diferença entre a Arquitetura Monolítica e a Arquitetura de Microserviços?

- A) A arquitetura monolítica só pode ser usada com bancos de dados NoSQL.
- B) No monolito, toda a aplicação está em uma única unidade; nos microserviços, cada funcionalidade é um serviço independente que pode usar tecnologias diferentes.**

- C) Microsserviços são sempre mais simples de implementar e devem ser usados em qualquer projeto.
- D) O monolito é exclusivo para aplicações frontend; microsserviços para backend.

✓ **Gabarito:** B

📖 **Explicação:** Monolito: única aplicação com todas as regras, dados e interfaces. Simples no início, difícil de escalar em sistemas grandes. Microsserviços: múltiplos serviços independentes, cada um com seu banco e tecnologia. Escala granular, mas com maior complexidade operacional (Kubernetes, CI/CD, monitoramento).

✗ **Por que as outras estão erradas:** A: monolito é independente do tipo de banco; C: microsserviços são indicados para sistemas grandes/complexos, não para qualquer projeto; D: ambos são para sistema completo.

💡 *Para sistemas pequenos/equipes reduzidas → Monolito. Para sistemas grandes/equipes distribuídas → Microsserviços.*

Questão 32 —

[REVISÃO] Qual método de avaliação arquitetural analisa os impactos das decisões de arquitetura nos atributos de qualidade ANTES da implementação?

- A) TDD (Test-Driven Development)
- B) ATAM (Architecture Tradeoff Analysis Method)**
- C) SCRUM
- D) Clean Architecture
- E) SOLID

✓ **Gabarito:** B

📖 **Explicação:** O ATAM (Architecture Tradeoff Analysis Method) permite avaliar a arquitetura ANTES de construir o sistema, identificando riscos, trade-offs e pontos fracos nos atributos de qualidade. Complementa-se com o SAAM, que foca em manutenibilidade. São as técnicas formais de avaliação arquitetural cobertas na disciplina.

✗ **Por que as outras estão erradas:** A: TDD é técnica de desenvolvimento com testes; C: Scrum é metodologia ágil de gestão; D: Clean Architecture é estilo de organização de código; E: SOLID são princípios de design orientado a objetos.

💡 *ATAM = Análise de Trade-offs Arquiteturais. Avalia ANTES de implementar. SAAM = foco em manutenibilidade.*

Questão 33 —

No arquivo docker-compose.yml, ao configurar o serviço de banco de dados, qual configuração é essencial para garantir que os dados não sejam perdidos entre reinicializações do contêiner?

- A) A configuração de porta (ports) para expor o banco externamente.
- B) A configuração de volumes para mapear o diretório de dados do banco para o sistema host.**
- C) A configuração de redes (networks) para conectar o banco ao back-end.
- D) A configuração de variáveis de ambiente (environment) com usuário e senha.

✓ **Gabarito:** B

 **Explicação:** A configuração de volumes é essencial para persistência. Sem ela, ao remover ou recriar o contêiner do banco, todos os dados se perdem. Com volumes, o diretório de dados do banco (ex: /var/lib/mysql) é mapeado para uma pasta no host, persistindo os dados independentemente do ciclo de vida do contêiner.

✗ Por que as outras estão erradas: A: portas são para acesso externo, não persistência; C: networks garantem comunicação, não persistência; D: variáveis de ambiente configuram acesso, não persistem dados.

 *volumes: = persistência no docker-compose.yml. Sem isso, o banco perde dados ao reiniciar!*

Questão 34 —

Qual padrão de design é mais adequado para implementar um sistema de notificações onde, ao concluir uma compra, múltiplos sistemas (e-mail, SMS, atualização de estoque, analytics) precisam ser acionados de forma desacoplada?

A) Singleton — para garantir que apenas um notificador exista.

B) Factory — para criar instâncias dos diferentes sistemas de notificação.

C) Observer / Pub-Sub — para que os sistemas se inscrevam e sejam notificados automaticamente quando o evento de compra ocorrer.

D) Strategy — para alternar entre diferentes algoritmos de notificação em tempo de execução.

 **Gabarito:** C

 **Explicação:** O Observer (Publish/Subscribe) é ideal quando múltiplos componentes independentes precisam reagir ao mesmo evento. O sistema de compras (publicador) emite o evento 'compra finalizada'; cada sistema de notificação (assinante) reage de forma independente. Isso garante baixo acoplamento — adicionar novo sistema de notificação não exige mudar o código de compra.

✗ Por que as outras estão erradas: A: Singleton não gerencia notificações; B: Factory criaria objetos, mas não gerencia o fluxo de eventos; D: Strategy é para algoritmos alternativos, não para múltiplos receptores simultâneos.

 *1 evento → N reações independentes = Observer. Compra finalizada → e-mail + SMS + estoque = Observer clássico!*

Questão 35 —

Analisando a estrutura de projeto docker-compose com Front-End (nginx:alpine), Back-End (node:alpine) e Banco de Dados (mongo), qual sequência descreve corretamente o estado (efêmero vs persistente) de cada contêiner?

A) Front-End: Persistente, Back-End: Efêmero, Banco de Dados: Efêmero

B) Todos os três contêineres são efêmeros pois usam imagens Alpine.

C) Front-End: Efêmero, Back-End: Efêmero, Banco de Dados: Necessita de Persistência

D) Front-End: Persistente, Back-End: Persistente, Banco de Dados: Efêmero

 **Gabarito:** C

 **Explicação:** Front-End e Back-End são efêmeros (stateless): podem ser recriados facilmente sem perda de dados, pois código e dependências estão na imagem. O Banco de Dados **NECESSITA** de persistência: contém os dados da aplicação que precisam sobreviver ao ciclo de vida do contêiner, exigindo volumes.

✗ Por que as outras estão erradas: A e D: invertidas a classificação do banco; B: Alpine é imagem leve, não determina o estado de persistência.

💡 *Stateless = efêmero (pode morrer e renascer). Stateful = precisa de volume. BD é sempre stateful!*

6. PRINCIPAIS ERROS QUE OS ALUNOS COMETEM

✗ Confundir Factory com Factory Method

☒ Correto: Factory Method é o padrão GoF (Criacional). 'Factory' é uma simplificação. Em provas, quando perguntado sobre a categoria, a resposta é CRIACIONAL.

✗ Confundir Observer com Strategy

☒ Correto: Observer = múltiplos componentes notificados (1:N). Strategy = trocar algoritmo em tempo de execução (1:1). Jornal+assinantes = Observer. GPS com rotas alternativas = Strategy.

✗ Achar que Alpine é o que torna contêineres leves

☒ Correto: O que torna contêineres leves é o COMPARTILHAMENTO DO KERNEL. Alpine é uma distro leve, mas é opcional.

✗ Confundir Docker Compose com Kubernetes

☒ Correto: Compose = orquestração simples, desenvolvimento. Kubernetes = orquestração de produção em escala.

✗ Achar que Front-end acessa diretamente o Banco de Dados

☒ Correto: NUNCA. O Back-end é sempre o intermediário. Front → Back → DB.

✗ Colocar regras de negócio no Controller (MVC)

☒ Correto: Regras de negócio ficam no MODEL. Controller no MVC é apenas intermediário.

✗ Confundir Adapter com Facade

☒ Correto: Adapter = compatibilidade de interfaces incompatíveis. Facade = simplificação de subsistema complexo.

✗ Achar que Volume é usado para comunicação entre contêineres

☒ Correto: Volume = persistência de dados. Comunicação entre contêineres = rede interna Docker.

✗ Confundir Repository com Singleton

☒ Correto: Repository = abstração do banco. Singleton = instância única. No projeto, o repositório é exportado como instância única, mas são padrões diferentes.

✗ Marcar 'Arquitetural' como categoria de padrão GoF

☒ Correto: GoF tem 3 categorias: Criacional, Estrutural, Comportamental. 'Arquitetural' não existe como categoria GoF.

7. REVISÃO RÁPIDA PARA A PROVA

Respostas-Relâmpago — Padrões de Design

Pergunta-relâmpago	Resposta
Factory Method pertence a qual categoria?	CRIACIONAL
Qual padrão usa a analogia do jornal com assinantes?	OBSERVER
Qual padrão converte interface incompatível?	ADAPTER
Qual padrão garante uma única instância global?	SINGLETON
Qual padrão troca algoritmos em tempo de execução?	STRATEGY
Quais são os 5 padrões mais usados no frontend JS?	Singleton, Observer, Strategy, Decorator, Module
Qual é a diferença entre Padrão de Projeto e Arquitetural?	Projeto = nível de código; Arquitetural = nível de sistema
Quais são as 3 categorias GoF?	Criacional, Estrutural, Comportamental
O Command encapsula o quê?	Uma ação em um objeto executável
Para que serve o Repository?	Abstrai o acesso a dados, isolando lógica do banco

Respostas-Relâmpago — Docker e Contêineres

Pergunta-relâmpago	Resposta
Por que contêineres são Megabytes e rápidos?	Compartilham o kernel do SO do host
Front-end usa qual imagem?	nginx:alpine — classificação: ESTÁTICA
Back-end usa qual imagem?	node:alpine — classificação: DINÂMICA
Banco de dados usa qual imagem?	mongo/mysql/postgres — classificação: PERSISTENTE
O que Docker Compose faz?	Define e executa múltiplos contêineres de forma integrada
Para que serve um volume?	Persistir dados que não se percam ao recriar o contêiner
Qual contêiner DEVE ter volume?	Banco de Dados (obrigatório)
O Front-end pode acessar o DB diretamente?	NÃO. Sempre passa pelo Back-end.
Imagem vs. Contêiner?	Imagem = molde (classe); Contêiner = instância em execução
Compose vs. Kubernetes?	Compose = dev/simples; K8s = produção/escala

8. MAPA MENTAL (FORMATO TEXTUAL)

ARQUITETURAS DE SOFTWARE

PADRÕES DE DESIGN

- └ Definição: soluções gerais e testadas para problemas recorrentes
- └ Origem: GoF 1994 – Design Patterns
- └ CRIACIONAIS (como criar objetos)
 - └ Singleton ← uma única instância global
 - └ Factory Method ← centraliza criação de objetos
 - └ Builder, Abstract Factory, Prototype
- └ ESTRUTURAIS (como objetos se organizam)
 - └ Adapter ← compatibilidade de interfaces
 - └ Facade ← interface simplificada de subsistema
 - └ Decorator ← adiciona comportamento dinamicamente
 - └ Proxy, Composite
- └ COMPORTAMENTAIS (comunicação entre objetos)
 - └ Observer ← notificação 1:N (jornal+assinantes)
 - └ Strategy ← algoritmos intercambiáveis
 - └ Command ← ação encapsulada em objeto
 - └ Repository ← abstração do acesso a dados
 - └ Module, State, Chain of Responsibility
- └ Frontend JS: Singleton, Observer, Strategy, Decorator, Module

DOCKER E CONTÊINERES

- └ Contêiner vs VM: compartilha kernel → menor + mais rápido
- └ Imagem = molde imutável; Contêiner = instância em execução
- └ Dockerfile = script para construir imagem
- └ Registry = repositório de imagens (Docker Hub)
- └ 3 CONTÊINERES
 - └ Front-End (nginx:alpine) – ESTÁTICO – sem volume
 - └ Back-End (node:alpine) – DINÂMICO – volume opcional
 - └ Banco de Dados (mongo/mysql) – PERSISTENTE – volume OBRIGATÓRIO
- └ Docker Compose: orquestra múltiplos contêineres + rede interna
- └ Volume: persistência de dados além do ciclo de vida do contêiner
- └ Fluxo: Front → Back → Banco → Back → Front

REVISÃO – FUNDAMENTOS

- └ Arquitetura = estrutura fundamental (componentes + relações + princípios)
- └ MVC: Model (lógica+dado) / View (interface) / Controller (intermediário)
- └ Microserviços: serviços pequenos, independentes, comunicam via API
- └ Teorema CAP: Consistência + Disponibilidade + Tolerância a Partições (só 2/3)
- └ Circuit Breaker: evita falhas em cascata
- └ API Gateway: roteamento central para microserviços
- └ ATAM: avaliação de trade-offs arquiteturais antes da implementação

9. CHECKLIST FINAL DE REVISÃO

Marque cada item após revisar:

- ☐ Sei definir e exemplificar os 3 tipos de padrões GoF (Criacional, Estrutural, Comportamental)
- ☐ Sei classificar corretamente: Factory Method → Criacional; Adapter → Estrutural; Observer → Comportamental
- ☐ Sei a analogia do Observer (jornal + assinantes) e do Adapter (adaptador de tomada)
- ☐ Sei os 5 padrões mais usados no frontend JS: Singleton, Observer, Strategy, Decorator, Module
- ☐ Sei a diferença entre Padrão de Projeto (código) e Padrão Arquitetural (sistema)
- ☐ Conheço o projeto pedido-simples: Singleton (config) → Strategy (desconto) → Factory (produto) → Command → Repository → Observer
- ☐ Sei por que contêineres são leves: COMPARTILHAMENTO DO KERNEL do SO host
- ☐ Sei diferenciar Imagem (molde) de Contêiner (instância em execução)
- ☐ Sei classificar os 3 contêineres: Front-End (Estático/Nginx), Back-End (Dinâmico/Node), BD (Persistente/mongo)
- ☐ Sei por que o BD precisa de volume: dados não se perdem ao reiniciar o contêiner
- ☐ Sei a função do Docker Compose: orquestrar múltiplos contêineres + criar rede interna
- ☐ Sei o fluxo correto: Front-End → Back-End → Banco de Dados (nunca Front → BD diretamente)
- ☐ Sei a diferença entre Docker Compose (dev) e Kubernetes (produção)
- ☐ Sei o Teorema CAP: Consistência, Disponibilidade, Tolerância a Partições (só 2/3)
- ☐ Sei onde ficam as regras de negócio no MVC: no MODEL
- ☐ Conheço o ATAM como método de avaliação arquitetural
- ☐ Sei que 'Arquitetural' NÃO é uma das 3 categorias GoF
- ☐ Sei que o Front-end nunca acessa o BD diretamente
- ☐ Sei que volumes NÃO são para comunicação (redes Docker fazem isso)
- ☐ Revisei os principais erros e sei evitá-los

Bons estudos e boa prova! 🍀

Prof. Esp. Leonardo Dias | UniFOA | Arquiteturas de Software